Raspberry Pi 5 NAS & Backup – Master Cheat Sheet

Here is your complete, step-by-step master cheat sheet for your Raspberry Pi 5 NAS and Backup environment.

This guide ensures that if a drive dies, you can drop in a new one, configure it, and seamlessly integrate it back into your existing Docker and Home Assistant infrastructure **without modifying your existing scripts, automations, or Docker Compose files.**

⚠ Prerequisite for Any Replacement

When you buy a new drive, its hardware ID (`UUID`) will be different. To ensure your existing scripts (like `nas-backup.sh`) and automations require **zero code changes**, you must update your system's mapping files (`/etc/crypttab` and `/etc/fstab`) with the new drive's UUID. This tricks the Pi into treating the new drive exactly like the old one.

Find your new drive letter (e.g., `sdd` or `sdc`) by running: `lsblk -f`

🔴 SCENARIO 1: NAS Drive Fails (Replacement & Restoration)

Assume your new NAS drive shows up as `/dev/sdX`. Replace `sdX` with your actual drive letter.

Step 1: Setup LUKS Encryption on the New Drive

```
sudo cryptsetup luksFormat /dev/sdX
```

(Enter the exact same encryption password you used previously.)

Step 2: Open the Encrypted Drive Check your `/etc/crypttab` for the exact mapper name you currently use (e.g., `nas_crypt`). Use that exact name:

```
sudo cryptsetup open /dev/sdX nas_crypt
```

Step 3: Format the Decrypted Volume as Btrfs

```
sudo mkfs.btrfs -L NAS_HDD /dev/mapper/nas_crypt
```

Step 4: Update System Mappings (The "Zero Code Change" Secret)

1. Get the UUID of the raw encrypted drive for `crypttab`: `sudo blkid /dev/sdX`
2. Get the UUID of the decrypted Btrfs volume for `fstab`: `sudo blkid /dev/mapper/nas_crypt`
3. Edit your crypttab and fstab files:

   ```
   sudo nano /etc/crypttab
   sudo nano /etc/fstab
   ```

   Replace the old NAS UUIDs in those files with the new ones you just copied. Save and exit.

Step 5: Mount & Restore Data

```
# Mount the new NAS drive using the updated fstab
sudo mount -a

# Restore data from the Backup drive to the NAS drive
# Using rsync ensures files are perfectly copied without carrying over read-only snapshot locks
sudo rsync -avHAX --progress /mnt/backup/latest_snapshot/ /mnt/nas/
```

Once the `rsync` command finishes, your NAS is fully restored. Start your Docker containers, and they will operate exactly as they did before the failure.

🔵 SCENARIO 2: Backup Drive Fails (Replacement & Setup)

Assume your new Backup drive shows up as `/dev/sdY`. Replace `sdY` with your actual drive letter.

Step 1: Setup LUKS Encryption

```
sudo cryptsetup luksFormat /dev/sdY
```

Step 2: Open the Encrypted Drive Use your standard backup mapper name (e.g., `backup_crypt`):

```
sudo cryptsetup open /dev/sdY backup_crypt
```

Step 3: Format the Volume as Btrfs

```
sudo mkfs.btrfs -L Backup_HDD /dev/mapper/backup_crypt
```

Step 4: Update System Mappings

1. `sudo blkid /dev/sdY` (For crypttab)
2. `sudo blkid /dev/mapper/backup_crypt` (For fstab)
3. Edit the files and replace the old Backup UUIDs:

```
sudo nano /etc/crypttab
sudo nano /etc/fstab
```

Step 5: Mount & Run Backup

```
sudo mount -a
```

Now, simply go to your Home Assistant dashboard and click your **NAS Backup** button. Because the mount paths (`/mnt/backup`) remain identical, `run_nas.sh` will instantly trigger the Btrfs snapshot backup on the new drive without a single code adjustment.

📂 SCENARIO 3: Accessing & Viewing Backup Files Anytime

Option A: Viewing files locally via terminal If the drive is already mounted via `fstab`, you can simply browse it:

```
cd /mnt/backup
ls -la
```

Option B: Exposing Backup Drive over Network (Samba Share) To view your backup files from your Windows/Mac/Linux machines over the network securely:

1. Open your Samba configuration:

```
sudo nano /etc/samba/smb.conf
```

2. Paste this block at the very bottom of the file (adjust the `path` if yours is different):

```
[Backup_HDD_Read_Only]
path = /mnt/backup
read only = yes
guest ok = no
valid users = redwannabil
browseable = yes
```

3. Restart Samba to apply changes:

```
sudo systemctl restart smbd
```

You can now safely open the Backup drive over your network. It is strictly forced to `read only = yes`, so you cannot accidentally delete or corrupt your backup snapshots from your computer.

🛠️ SCENARIO 4: NAS/Backup Troubleshooting & Maintenance

1. Detect Bit Rot & Repair Data (Btrfs Scrub) Because you are using Btrfs, you can run a background scrub to read every block of data, verify it against its checksum, and auto-repair corruption.

```
# Scrub the NAS drive
sudo btrfs scrub start /mnt/nas

# Check the status of the scrub
sudo btrfs scrub status /mnt/nas
```

2. Check Physical Drive Health (SMART Data) If a drive starts acting slow or throwing errors, check its physical hardware health.

```
sudo smartctl -a /dev/sdX
```

Look for `Reallocated_Sector_Ct` or `UDMA_CRC_Error_Count`. If these numbers are rising, the physical drive is dying.

3. Fix "Drive is locked / Read-Only" Errors If your drive suddenly turns read-only, it means Btrfs detected a severe hardware/I/O error and locked itself to prevent data loss. Check the kernel logs immediately to see the exact hardware error:

```
sudo journalctl -k -n 100 | grep -i btrfs
```

or

```
dmesg -T | tail -n 50
```

4. Btrfs Filesystem Repair (Use Only if Unmountable) If the Btrfs filesystem crashes and refuses to mount, DO NOT run standard `fsck`. Use the Btrfs rescue tools:

```
# Clear the log tree (fixes 90% of mount failures)
sudo btrfs rescue zero-log /dev/mapper/nas_crypt

# Only if the above fails, attempt a heavy repair (Dangerous)
sudo btrfs check --repair /dev/mapper/nas_crypt
```

5. Verify LUKS Encryption Status If a drive isn't mounting properly on boot, check if LUKS is successfully decrypting it:

```
sudo cryptsetup status nas_crypt
```